

BÁO CÁO LAB 4
MÔN HỌC MÁY HỌC THỐNG KÊ
Chủ đề: Cài đặt Decision Tree và Random Forest
Dataset: Wine Quality

Thông tin	Nội dung
Họ và tên	Bùi Thị Thanh Tuyền
MSSV	24521956
Lớp	DS102.Q21.2

Năm học: 2025-2026

1. Mục tiêu bài lab

Bài lab yêu cầu cài đặt và đánh giá hai mô hình học máy dạng cây là Decision Tree và Random Forest. Trong đó, hai mô hình tự cài đặt được xây dựng bằng NumPy, sau đó được so sánh với phiên bản sử dụng thư viện scikit-learn.

- Assignment 1: cài đặt Decision Tree bằng NumPy và đánh giá trên Wine Quality dataset bằng F1-score.
- Assignment 2: cài đặt Random Forest bằng NumPy và đánh giá trên cùng dataset bằng F1-score.
- Assignment 3: huấn luyện và đánh giá Decision Tree, Random Forest bằng thư viện học máy.

Assignment 1:

- Implementing Decision Tree using NumPy.
- Train and evaluate this method on the [Wine Quality](#) dataset using F1 score.

Assignment 2:

- Implementing Random Forest using NumPy.
- Train and evaluate this method on the [Wine Quality](#) dataset using F1 score.

Assignment 3:

- Train and evaluate the Decision Tree method using a machine learning library.
- Train and evaluate the Random Forest method using a machine learning library.

Hình 1. Yêu cầu của bài lab

2. Mô tả dữ liệu và tiền xử lý

Dữ liệu sử dụng là Wine Quality dataset gồm hai tập dữ liệu rượu vang đỏ và rượu vang trắng. Hai tập dữ liệu được gộp lại thành một bảng chung, đồng thời thêm thuộc tính type để phân biệt loại rượu.

Đặc điểm	Giá trị trong thực nghiệm
Số mẫu	6497
Số thuộc tính đầu vào	12
Số lớp sau khi mã hóa	7
Tập huấn luyện / kiểm tra	5197 / 1300
Nhãn dự đoán	quality
Chỉ số đánh giá chính	Macro F1-score

Quy trình tiền xử lý được thực hiện trong file main.py:

- Đọc hai file winequality-red.csv và winequality-white.csv
- Thêm biến type: rượu vang đỏ được gán 0, rượu vang trắng được gán 1.
- Gộp hai tập dữ liệu bằng pandas.concat.
- Tách X bằng cách loại cột quality; y là cột quality.
- Mã hóa nhãn quality về dạng số nguyên liên tiếp bằng LabelEncoder.
- Chia train/test theo tỉ lệ 80/20, có stratify theo nhãn để giữ phân bố lớp tương đối ổn định.

3. Phương pháp cài đặt

3.1. Decision Tree tự cài đặt bằng NumPy

Mô hình Decision Tree được cài đặt theo hướng đệ quy. Tại mỗi node, thuật toán tìm thuộc tính và ngưỡng chia tốt nhất dựa trên độ đo Gini impurity. Nếu node không còn đủ điều kiện chia tiếp, node trở thành lá và dự đoán lớp xuất hiện nhiều nhất.

Công thức Gini impurity: $Gini(S) = 1 - \sum(p_k^2)$, trong đó p_k là tỉ lệ mẫu thuộc lớp k trong tập S .

Gini sau khi chia: $Gini_split = (n_left/n)Gini(left) + (n_right/n)Gini(right)$.

- Mỗi thuộc tính được sắp xếp theo giá trị tăng dần.
- Các ngưỡng ứng viên được lấy là trung điểm giữa hai giá trị liên tiếp khác nhau.
- Chọn phép chia có weighted Gini nhỏ nhất.
- Điều kiện dừng gồm `max_depth`, `min_samples_split`, `min_samples_leaf` và node thuần nhất.

```
# Ý tưởng chính khi dự đoán một mẫu
if node is leaf:
    return node.class
if x[node.feature] < node.threshold:
    go to left child
else:
    go to right child
```

3.2. Random Forest tự cài đặt bằng NumPy

Random Forest được xây dựng bằng cách huấn luyện nhiều Decision Tree trên các bootstrap sample khác nhau. Mỗi cây chỉ xét một tập con ngẫu nhiên các thuộc tính tại mỗi lần tìm split, giúp tăng sự đa dạng giữa các cây.

- Tạo `n_estimators` cây quyết định.
- Với mỗi cây, lấy mẫu bootstrap từ tập huấn luyện với kích thước bằng tập huấn luyện ban đầu.
- Mỗi cây sử dụng `max_features="sqrt"` để chọn ngẫu nhiên một phần thuộc tính khi split.
- Kết quả dự đoán cuối cùng được lấy bằng majority voting giữa các cây.

```
# Ý tưởng majority voting
predictions = [tree.predict(X) for tree in forest]
final_prediction = most_common_class(predictions)
```

3.3. Mô hình sử dụng thư viện scikit-learn

Để đối chiếu với mô hình tự cài đặt, bài lab sử dụng `DecisionTreeClassifier` và `RandomForestClassifier` của scikit-learn với các siêu tham số tương tự như phiên bản NumPy.

- Decision Tree: `max_depth=100`, `min_samples_split=5`, `min_samples_leaf=2`.
- Random Forest: `n_estimators=50`, `max_depth=100`, `min_samples_split=5`, `min_samples_leaf=2`, `max_features="sqrt"`.

4. Chỉ số đánh giá

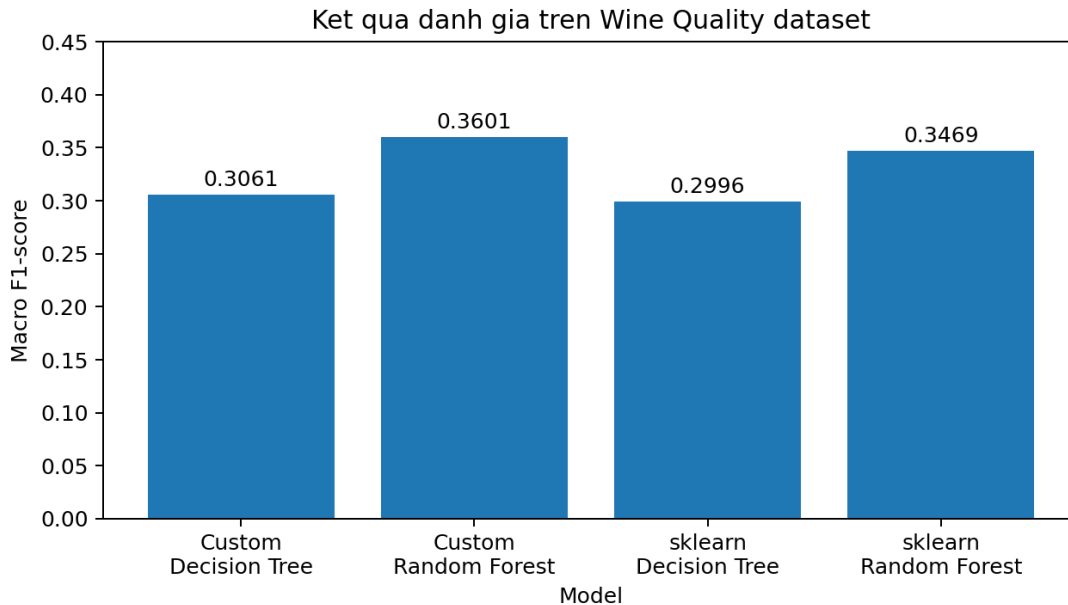
Bài toán Wine Quality là bài toán phân loại đa lớp. Do phân bố nhãn trong dataset thường không đều, báo cáo chọn Macro F1-score làm chỉ số chính để đánh giá công bằng hơn giữa các lớp.

F1-score của một lớp: $F1 = 2 * Precision * Recall / (Precision + Recall)$.

Macro F1-score: tính F1 riêng cho từng lớp rồi lấy trung bình đều giữa các lớp. Nhờ đó, các lớp ít mẫu không bị che khuất bởi các lớp nhiều mẫu.

5. Kết quả thực nghiệm

Mô hình	Cách cài đặt	Macro F1-score
Decision Tree	NumPy tự cài đặt	0.3061
Random Forest	NumPy tự cài đặt	0.3601
Decision Tree	scikit-learn	0.2996
Random Forest	scikit-learn	0.3469



Hình 2. So sánh Macro F1-score giữa các mô hình

Output chương trình:

```
Dataset : 6497 samples, 12 features
Classes : 7 (quality scores re-encoded 0-based)
Train/Test split: 5197 / 1300
```

```
Assignment 1: Custom Decision Tree      Macro F1-score: 0.3061
Assignment 2: Custom Random Forest      Macro F1-score: 0.3601
Assignment 3: sklearn Decision Tree      Macro F1-score: 0.2996
Assignment 3: sklearn Random Forest      Macro F1-score: 0.3469
```

6. Nhận xét và thảo luận

- Random Forest cho kết quả tốt hơn Decision Tree ở cả hai nhóm cài đặt. Đây là xu hướng hợp lý vì Random Forest kết hợp nhiều cây, giúp giảm phương sai so với một cây đơn lẻ.
- Custom Decision Tree có Macro F1-score 0.3061, gần với scikit-learn Decision Tree 0.2996. Điều này cho thấy cách cài đặt NumPy hoạt động đúng về mặt ý tưởng.
- Custom Random Forest đạt 0.3601, cao hơn scikit-learn Random Forest 0.3469 trong lần chạy này. Sự khác biệt có thể đến từ quá trình lấy mẫu ngẫu nhiên, cách chọn split và seed ngẫu nhiên.
- Macro F1-score tương đối thấp vì bài toán nhiều lớp và các lớp ít mẫu khó dự đoán hơn. Khi dùng Macro F1, lỗi trên lớp thiểu số có ảnh hưởng rõ hơn đến điểm tổng thể.